
Modelo Probabilístico: BM25

Aula 04 — Saturação, tamanho de documento e ranqueamento

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Fase 1 (Aulas 01–03)

- Texto → termos → índice
- Espaço vetorial e cosseno
- TF-IDF como peso

Fase 2 começa hoje

- Modelo **probabilístico**
- **BM25**: o padrão da indústria
- Depois: avaliação e realimentação

Meta da aula

Entender *por que* o BM25 ranqueia melhor que o TF-IDF e implementá-lo em R.

Por que ir além do TF-IDF?

- No TF-IDF, a frequência cresce **linearmente**: a 10ª ocorrência vale tanto quanto a 2ª.
- Documentos **longos** tendem a acumular termos e inflar o escore.
- Falta um tratamento **princiado** de frequência e de tamanho.

Ideia probabilística

Ranquear pela **probabilidade de relevância** $P(R \mid d, q)$. O BM25 é a forma prática e vencedora dessa família.

Antes da fórmula: de onde ela vem

Não vamos apresentar o BM25 pronto. Vamos **construí-lo** — partindo de uma pergunta estatística simples.

A pergunta

Quantas vezes esperamos que uma palavra apareça num documento? E o que a **contagem observada** nos diz sobre o documento ser ou não sobre aquele assunto?

O caminho

- ① A distribuição de **Poisson** e para que serve.
- ② Por que uma Poisson **só** não descreve bem as palavras.
- ③ A ideia de **eliteness**: duas Poissons.
- ④ O peso do termo, e a saturação **emergindo** do modelo.
- ⑤ A aproximação que **produz** o BM25.

A distribuição de Poisson

Modela a **contagem de eventos raros** num intervalo fixo, quando os eventos são independentes e ocorrem a uma taxa média constante.

$$P(X = f) = \frac{e^{-\lambda} \lambda^f}{f!}$$

- X : a **variável** — quantas vezes o evento ocorreu.
- f : um **valor** específico dessa contagem $(0, 1, 2, \dots)$.
- λ : a **taxa média** de ocorrências no intervalo.

É o único parâmetro: ele determina toda a distribuição.

Exemplos clássicos: chamadas por hora num *call center*, defeitos por metro de tecido — e **ocorrências de uma palavra por documento**.

Suponha que a palavra “praia” apareça em média **2 vezes** por documento ($\lambda = 2$):

f	0	1	2	3	5
$P(X = f)$	0,135	0,271	0,271	0,180	0,036

- A distribuição tem **pico** perto da média e cai rápido nos extremos.
- Ver a palavra **5 vezes** teria probabilidade de apenas 3,6%.

Guarde essa última linha: é exatamente onde o modelo vai **falhar**.

O problema de informação: palavras são “explosivas”

Se cada palavra fosse sorteada independentemente, uma Poisson bastaria. Mas texto real não é assim.

A observação (*burstiness*)

Palavras de **conteúdo** não se espalham uniformemente: ou o documento **não fala** do assunto (zero ocorrências), ou fala e então a palavra aparece **muitas vezes**.

- “praia” num artigo de economia: 0 vezes.
- “praia” num artigo sobre o litoral: 8, 12, 20 vezes.
- O caso intermediário — exatamente 2 — é **raro**.

A Poisson única prevê o oposto: muitos casos intermediários e quase nenhum extremo.

Quanto a Poisson única erra

Compare uma **mistura** (15% dos documentos com $\lambda = 4$, o resto com $\mu = 0,1$) com uma Poisson única de **mesma média** (0,685):

f	mistura	Poisson única	razão
0	0,772	0,504	1,5×
2	0,026	0,118	0,2×
4	0,029	0,005	6,3×
6	0,016	0,00007	216×

Leia a última linha

Ver a palavra **6 vezes** é 216× mais provável na realidade do que a Poisson única prevê. O modelo simples **não explica** os documentos que falam muito do assunto.

A saída: *eliteness* — duas populações

A hipótese

Para cada termo, os documentos se dividem em **duas classes ocultas**:

- **Elite** (E): o documento é genuinamente *sobre* aquele assunto.
- **Não-elite** ($\bar{E}$): o termo aparece de passagem.

Em **cada** classe, a contagem é Poisson — mas com taxas diferentes:

$$f \mid E \sim \text{Poisson}(\lambda), \quad f \mid \bar{E} \sim \text{Poisson}(\mu), \quad \lambda > \mu$$

- λ : taxa média **nos documentos elite** (alta).
- μ : taxa média **nos não-elite** (baixa, perto de zero).

“Elite” é uma classe **não observável**: não sabemos quais documentos são, só vemos a contagem.

O peso do termo: razão de verossimilhanças

No modelo probabilístico, o peso de um termo é o **log da razão** entre vê-lo em relevantes e em irrelevantes:

$$w(f) = \log \frac{P(f \mid R)}{P(f \mid \bar{R})}$$

Supondo que a relevância afeta f *apenas* através da eliteness, cada lado vira uma **mistura**:

$$w(f) = \log \frac{p e^{-\lambda} \lambda^f + (1 - p) e^{-\mu} \mu^f}{q e^{-\lambda} \lambda^f + (1 - q) e^{-\mu} \mu^f}$$

- $p = P(E \mid R)$: chance de ser elite **dado que é relevante**.
- $q = P(E \mid \bar{R})$: chance de ser elite **dado que não é**.
- Naturalmente $p > q$ — documentos relevantes têm mais chance de ser elite.

Os $f!$ cancelam entre numerador e denominador.

A saturação emerge do modelo

Analise os dois extremos dessa expressão:

Em $f = 0$

O peso é uma constante.
Normalizamos subtraindo-a, para que $w(0) = 0$.

Quando $f \rightarrow \infty$

O termo λ^f **domina** os dois lados (pois $\lambda > \mu$), e tudo colapsa em

$$w(\infty) \rightarrow \log \frac{p}{q}$$

O ponto central

A saturação **não foi imposta** — ela *cai* da estrutura do modelo. Por mais que o termo se repita, você não fica mais convencido de que “este documento é elite”, e essa convicção tem **teto**.

Calculando a saturação passo a passo

Com $\lambda = 3$, $\mu = 0,2$, $p = 0,6$, $q = 0,1$. Primeiro as constantes:

$$e^{-\lambda} = e^{-3} = 0,049787 \quad e^{-\mu} = e^{-0,2} = 0,818731$$

O valor em $f = 0$ (aqui $\lambda^0 = \mu^0 = 1$, sobram só os pesos)

$$\text{num} = 0,6 \times 0,049787 + 0,4 \times 0,818731 = 0,357365$$

$$\text{den} = 0,1 \times 0,049787 + 0,9 \times 0,818731 = 0,741836$$

$$w(0) = \log\left(\frac{0,357365}{0,741836}\right) = \log(0,481730) = -\mathbf{0,7304}$$

Repare: $w(0)$ é **negativo**. É por isso que precisamos normalizar.

A saturação: cuidado com a normalização

O valor “cru”

$$w(\infty) = \log \frac{p}{q} = \log \frac{0,6}{0,1} = \log 6 = \mathbf{1,7918}$$

O valor **normalizado** (que usamos na tabela)

$$w(\infty) - w(0) = 1,7918 - (-0,7304) = \mathbf{2,5221}$$

Se você calculou 1,79, o cálculo estava certo — faltou apenas **subtrair** $w(0)$. A normalização existe para que a ausência do termo ($f = 0$) contribua com **zero**, e não com um valor negativo.

A curva 2-Poisson em números

Aplicando a fórmula para cada f (já normalizado, isto é, $w(f) - w(0)$):

f	$w(f)$ normalizado	
0	0,000	ponto de partida
1	0,685	
2	2,064	sobe rápido
3	2,483	já quase no teto
5	2,522	
8	2,522	congelou

O teto é o **2,5221** que acabamos de calcular. Da 3ª ocorrência em diante, o termo praticamente não acrescenta mais evidência.

“Mas por que não usar essa fórmula direto?”

É a pergunta certa. Temos $w(f)$ exata — por que aproximar? Veja o que seria preciso saber, **para cada palavra do vocabulário**:

Parâmetro	O que é	Como estimar?
λ	taxa nos docs elite	preciso saber <i>quais são</i> elite
μ	taxa nos não-elite	idem
p	$P(E \mid R)$	preciso de julgamentos de relevância
q	$P(E \mid \bar{R})$	idem

O impasse

Eliteness é uma classe oculta: para estimar λ e μ eu precisaria saber quem é elite — mas é justamente isso que estou tentando inferir. E p e q exigem julgamentos de relevância que, numa busca nova, **não existem**.

Quatro parâmetros ocultos, por termo

Mesmo que fosse possível estimar, o custo seria proibitivo:

- Vocabulário da Wikipédia (pt): **~ 2 milhões** de termos distintos.
- 4 parâmetros por termo → **8 milhões** de parâmetros a estimar.
- Cada um com **poucos dados** (termos raros aparecem em 2 ou 3 documentos).
- Resultado: estimativas com **variância enorme** — o modelo decoraria ruído.

O dilema clássico

A fórmula exata é **teóricamente correta** e **praticamente inútil**.

Precisamos de algo que preserve o *comportamento* sem exigir os parâmetros.

A saída: copiar o formato, não a fórmula

Robertson observou que, para **ranquear**, não importa reproduzir os valores exatos de $w(f)$ — importa preservar seu **comportamento qualitativo**:

Propriedade	Por quê
$w(0) = 0$	termo ausente não contribui
crescente	mais ocorrências, mais evidência
côncava	cada ocorrência nova vale menos
satura em valor finito	a convicção tem teto

A troca

Qualquer função com essas quatro propriedades produz **a mesma ordenação** — e podemos escolher a mais simples, com **um** parâmetro ajustável em vez de quatro ocultos.

A hipérbole normalizada reproduz o formato com **um** parâmetro em vez de quatro:

$$w(f) \approx w_{\infty} \cdot \frac{f}{f + k_1}$$

E o que é w_{∞} , o peso de saturação? É a importância máxima do termo — estimada, sem julgamentos de relevância, pelo **IDF**. Normalizando para $w(1) = \text{IDF}$:

$$w(f) \approx \text{IDF}(t) \cdot \frac{f(k_1 + 1)}{f + k_1}$$

Este é o **núcleo do BM25**. Falta ainda corrigir pelo **tamanho** do documento — e aí teremos a fórmula completa.

Uma honestidade sobre a aproximação

- A curva 2-Poisson tem um “joelho” **abrupto** (salta de 0,69 para 2,06 entre $f=1$ e $f=2$); a hipérbole é mais **suave**.
- O erro entre as duas **não é pequeno**.

Por que ainda assim vale

A aproximação não é boa por ser *precisa* — é boa por ser **robusta**. Captura o comportamento qualitativo certo com **um** parâmetro ajustável empiricamente por *corpus*.

Trocar fidelidade teórica por estabilidade prática: é essa a escolha de engenharia que o BM25 representa.

(E explica o nome: *Best Matching* **25** — a 25ª tentativa.)

A fórmula do BM25: não se assuste

$$\text{BM25}(q, d) = \underbrace{\sum_{t \in q}}_{\text{1. soma os termos}} \underbrace{\text{IDF}(t)}_{\text{2. raridade}} \cdot \underbrace{\frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}}_{\text{3. frequência corrigida}}$$

Símbolo	Nome	O que é
q	consulta	o que o usuário digitou (<i>query</i>)
d	documento	um dos textos do <i>corpus</i>
t	termo	cada palavra da consulta; $t \in q$
$f_{t,d}$	frequência	quantas vezes t aparece em d
df_t	<i>document frequency</i>	em quantos documentos t aparece
N	tamanho do <i>corpus</i>	total de documentos
$ d $	tamanho do documento	quantas palavras tem d
avgdl	<i>average doc length</i>	tamanho médio dos documentos
k_1	parâmetro	controla a saturação (típico: 1,2)
b	parâmetro	controla o peso do tamanho (típico: 0,75)

Apenas k_1 e b são **escolhidos por você**; todo o resto vem do *corpus*.

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d} (k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}$$

Em palavras

“Para **cada termo** da consulta, multiplique o quanto ele é **raro** pelo quanto ele **aparece neste documento** — corrigindo essa frequência pela **saturação** e pelo **tamanho** do documento. Depois, **some tudo**.”

- 1. Mesma estrutura do TF-IDF: cada termo contribui e **somamos**.
- 2. O IDF continua lá, medindo raridade.
- 3. A fração é *toda* a novidade — vamos abri-la em duas partes.

Peça 3, parte A: a saturação (k_1)

Ignore o tamanho por um instante (ponha $b = 0$). Sobra:

$$\frac{f(k_1 + 1)}{f + k_1}$$

ocorrências f	TF-IDF	BM25	ganho
1	1	1,000	—
2	2	1,375	+0,375
3	3	1,571	+0,196
4	4	1,692	+0,121
5	5	1,774	+0,082

O TF-IDF cresce **sem limite**. O BM25 tende a um **teto** de $k_1 + 1 = 2,2$ — por mais que o termo se repita, o escore nunca passa disso.

O que k_1 realmente controla

f	$k_1 = 0$	$k_1 = 0,5$	$k_1 = 1,2$	$k_1 = 2,0$
1	1,000	1,000	1,000	1,000
3	1,000	1,286	1,571	1,800
5	1,000	1,364	1,774	2,143
teto	1,0	1,5	2,2	3,0

- $k_1 = 0$: a frequência é **ignorada** — vira busca booleana (tem ou não tem).
- k_1 **pequeno**: satura rápido; a 2ª ocorrência já quase não acrescenta.
- k_1 **grande**: satura devagar; aproxima-se do TF linear do TF-IDF.

k_1 é o botão que regula *quanto* a repetição ainda vale.

Peça 3, parte B: o tamanho (b)

Agora o denominador completo. Chamamos de K tudo o que multiplica no denominador:

$$K = k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right) \implies \frac{f(k_1 + 1)}{f + K}$$

- $|d|/\text{avgdl}$ compara o documento com a **média do corpus**.
- Documento **do tamanho médio** $\rightarrow$ razão = 1 $\rightarrow K = k_1$ (nada muda).
- b decide **quanto** essa comparação pesa: $b = 0$ ignora o tamanho; $b = 1$ normaliza totalmente.

K é apenas um **apelido** para o parêntese — guarde esse nome, ele reaparece no código.

Por que b interpola (e não normaliza sempre)

Um documento pode ser longo por **duas razões opostas**:

Verbosidade

Mesmo conteúdo, mais palavras.

O autor é prolixo.

⇒ **deve** normalizar ($b = 1$).

Escopo

Cobre **mais assuntos**.

Uma enciclopédia legítima.

⇒ **não** deve normalizar ($b = 0$).

O dilema

Olhando só o tamanho, **não dá para saber** qual dos dois casos é. Então b **interpola** entre as duas hipóteses — e 0,75 é o meio-termo que funciona empiricamente na maioria dos *corpora*.

O K em números, no nosso *corpus*

Com $k_1 = 1,2$, $b = 0,75$ e $\text{avgdl} = 8$, veja o **mesmo termo aparecendo uma vez** ($f = 1$) em documentos de tamanhos diferentes:

doc	$ d $	$ d /\text{avgdl}$	K	contribuição
d4	6	0,750	0,975	1,114
d1	7	0,875	1,087	1,054
d6	8	1,000	1,200	1,000
d3	9	1,125	1,312	0,951

Leia assim

O termo aparece **uma vez** nos quatro. Mas em d4 (curto) vale 1,114 e em d3 (longo) só 0,951. Documento longo tem mais chance de conter o termo **por acaso**.

O BM25 usa uma versão diferente do IDF da Aula 01:

$$\text{IDF}(t) = \log\left(\frac{N - \text{df}_t + 0,5}{\text{df}_t + 0,5} + 1\right)$$

- $N - \text{df}_t$: em quantos documentos o termo **não** aparece.
- df_t : em quantos ele **aparece**.
- A razão é, portanto, “**quem não tem**” dividido por “**quem tem**” — uma *razão de chances*. Quanto maior, mais raro o termo.

Mesma ideia de sempre (**raro pesa mais**), formulada de um jeito mais robusto.

Calculando passo a passo

Para modelo, que aparece em $df = 2$ dos $N = 8$ documentos:

- 1 Numerador: $8 - 2 + 0,5 = 6,5$
- 2 Denominador: $2 + 0,5 = 2,5$
- 3 Razão: $6,5 / 2,5 = 2,6$
- 4 Soma 1: $2,6 + 1 = 3,6$
- 5 Logaritmo: $\ln(3,6) = \mathbf{1,281}$

termo	df	$N - df + 0,5$	$df + 0,5$	razão	IDF
modelo	2	6,5	2,5	2,600	1,281
recuperacao	2	6,5	2,5	2,600	1,281
documentos	4	4,5	4,5	1,000	0,693
de	5	3,5	5,5	0,636	0,492

Para que servem o $+0,5$ e o $+1$

Os $+0,5$: suavização

Se um termo aparecesse em **todos** os documentos, teríamos $N - df = 0$ e o logaritmo explodiria para $-\infty$. Os $+0,5$ garantem que numerador e denominador **nunca zerem**.

O $+1$: impede IDF negativo

Sem ele, qualquer termo em **mais da metade** do *corpus* teria razão < 1 , logo logaritmo **negativo**.

No nosso caso, de ($df= 5$ de 8) ficaria com $-0,452$.

Um IDF negativo **puniria** o documento por conter o termo que o usuário buscou. Com o $+1$, o pior caso é um peso perto de zero: não ajuda, mas não atrapalha.

termo	df	clássico $\log(N/df)$	BM25
modelo	2	1,386	1,281
documentos	4	0,693	0,693
de	5	0,470	0,492

- Os valores são **próximos** e a **ordem é a mesma**: raro continua pesando mais.
- Curiosidade: quando $df = N/2$ (caso de documentos), as duas fórmulas dão **exatamente** $\ln 2$. Elas se cruzam nesse ponto.

Trocar uma pela outra raramente muda o ranking. A do BM25 é mais robusta nos **extremos**.

Retomando o *corpus* e a matriz de frequências

```
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "o indice invertido acelera a busca em muitos documentos",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
tok    <- function(x) unlist(strsplit(tolower(x), "\\s+"))
tokens <- lapply(docs, tok)
vocab  <- sort(unique(unlist(tokens)))
```

O mesmo pré-processamento da Aula 03 — agora precisamos também do **tamanho** de cada documento.

O que o BM25 exige: tf , $|d|$ e a média

```
# matriz termo-documento: linhas = termos, colunas = documentos
tf <- sapply(tokens, function(t) as.integer(table(factor(t, levels = vocab))))
rownames(tf) <- vocab

dl      <- colSums(tf)      # |d|: quantas palavras tem cada documento
avgdl   <- mean(dl)        # avgdl: tamanho MEDIO do corpus
dl
avgdl

d1 d2 d3 d4 d5 d6 d7 d8
 7  9  9  6  9  8  9  7
[1] 8
```

Os documentos têm de **6 a 9** palavras, com média **8**. É contra essa média que o parâmetro b vai comparar cada documento.


```
N <- ncol(tf)          # total de documentos (8)
df <- rowSums(tf > 0)  # em quantos docs cada termo aparece

# versao probabilistica do IDF: os "+0.5" evitam divisao/log por zero
idf <- log((N - df + 0.5) / (df + 0.5) + 1)

round(idf[c("modelo", "de", "recuperacao", "documentos")], 3)
```

modelo	de	recuperacao	documentos
1.281	0.492	1.281	0.693

Mesma ideia da Aula 01 — **raro pesa mais** — com a diferença de que o numerador conta os documentos que **não** têm o termo ($N - df$).

Implementando o BM25

```
k1 <- 1.2; b <- 0.75                                # parametros padrao

bm25_doc <- function(termos, d) {
  s <- 0                                              # acumulador do somatorio
  for (t in termos) {                                # SOMA sobre os termos da consulta
    if (!(t %in% vocab)) next                        # termo inexistente: contribui 0
    f <- tf[t, d]                                    # f: frequencia do termo NESTE doc
    K <- k1 * (1 - b + b * dl[d] / avgdl)           # penalidade de tamanho
    s <- s + idf[t] * (f * (k1 + 1)) / (f + K)
  }
  s
}
```

O for é literalmente o $\sum_{t \in q}$ da fórmula; o K é todo o parêntese do denominador.

A função, linha a linha

- ① `s <- 0` — o acumulador. O BM25 é uma **soma**, então começamos do zero.
- ② `for (t in termos)` — é o $\sum_{t \in q}$ da fórmula, escrito em R.
- ③ `if (!(t %in% vocab)) next` — termo que não existe no *corpus* contribui **zero**; pulamos.
- ④ `f <- tf[t, d]` — busca $f_{t,d}$ na matriz: linha do termo, coluna do documento.
- ⑤ `K <- k1 * (...)` — exatamente o K do slide anterior, calculado para este documento.
- ⑥ `s <- s + idf[t] * (...)` — a contribuição daquele termo, somada ao acumulador.

Cada linha corresponde a um pedaço da fórmula. Não há nada além disso.

```
consulta <- tok("modelo de recuperacao")  
scores <- sapply(colnames(tf), function(d) bm25_doc(consulta, d))  
round(sort(scores, decreasing = TRUE), 3)
```

d3	d1	d2	d4	d8	d6	d5	d7
1.873	1.869	1.687	1.427	0.519	0.492	0.000	0.000

O BM25 coloca d3 e d1 no topo — concordando com o cosseno da Aula 02, que liderava com d1.

Vendo a saturação de frequência

```
# quanto uma ocorrencia extra ainda contribui (sem normalizacao de tamanho)
sat <- function(f) (f * (k1 + 1)) / (f + k1)
round(sapply(1:5, sat), 3)
[1] 1.000 1.375 1.571 1.692 1.774
```

Da 1ª para a 2ª ocorrência o ganho é grande; depois, cada vez menor.

TF-IDF

- Frequência **linear**
- Normalização via cosseno
- Ótimo didático

BM25

- Frequência **saturada** (k_1)
- Tamanho **controlado** (b)
- *Baseline* forte até hoje

BM25 é o estágio 1 de recuperação em muitos sistemas modernos (antes do *rerank* neural).

Tarefa

- 1 Implementar o BM25 sobre o *corpus* de 8 documentos.
- 2 Comparar o *ranking* do BM25 com o do TF-IDF (Aula 02) para 3 consultas.
- 3 Variar k_1 e b e observar o efeito na ordem dos resultados.

Próxima aula

- Como **avaliar** um ranking
- P@k, MAP, nDCG, MRR

Leitura

- Manning et al., *IIR* — cap. 11
- Baeza-Yates & Ribeiro-Neto — cap. 3

Obrigado!

Dúvidas? Vamos ao R.
